Exercise 2

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

This **CLI (Command Line Interface)** Python program is a simple contact manager that lets users add, view, and delete contacts through a text-based menu in the terminal.

```
contacts = []

def add_contact():
    name = input("Enter contact name: ")
    phone = input("Enter phone number: ")
    contact = {"name": name, "phone": phone}
    contacts.append(contact)
    print(f"Contact '{name}' added successfully.")

def view_contacts():
    if contacts:
        print("\nYour Contacts:")
        for idx, contact in enumerate(contacts, 1):
            print(f"{idx}. {contact['name']} - {contact['phone']}")
    else:
        print("No contacts available.")

def delete_contact():
    view_contacts()
    if contacts:
```

```python
        try:
            choice = int(input("Enter contact number to delete: "))
            if 0 < choice <= len(contacts):
                removed = contacts.pop(choice - 1)
                print(f"Contact '{removed['name']}' deleted.")
            else:
                print("Invalid choice.")
        except ValueError:
            print("Please enter a valid number.")

def main():
    while True:
        print("\n==== Contact Manager ====")
        print("1. Add Contact")
        print("2. View Contacts")
        print("3. Delete Contact")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            add_contact()
        elif choice == '2':
            view_contacts()
        elif choice == '3':
            delete_contact()
        elif choice == '4':
            print("Exiting program...")
            break
        else:
            print("Invalid option. Try again.")

if __name__ == "__main__":
    main()
```

Sample Output:

```
--- Contact Manager CLI ---
1. Add Contact
2. View Contacts
3. Delete Contact
4. Exit
Enter your choice: 1
Enter contact name: Kumar
Enter phone number: 1234567890
Contact added successfully.

--- Contact Manager CLI ---
1. Add Contact
2. View Contacts
3. Delete Contact
4. Exit
Enter your choice: 2

Contacts List:
1. Kumar - 1234567890

--- Contact Manager CLI ---
1. Add Contact
2. View Contacts
3. Delete Contact
4. Exit
Enter your choice: 1
Enter contact name: Kishore
Enter phone number: 09887654321
Contact added successfully.
```

```
--- Contact Manager CLI ---
1. Add Contact
2. View Contacts
3. Delete Contact
4. Exit
Enter your choice: 3

Contacts List:
1. Kumar - 1234567890
2. Kishore - 09887654321
Enter contact number to delete: 2
Kishore deleted successfully.

--- Contact Manager CLI ---
1. Add Contact
2. View Contacts
3. Delete Contact
4. Exit
Enter your choice: 2

Contacts List:
1. Kumar - 1234567890
```

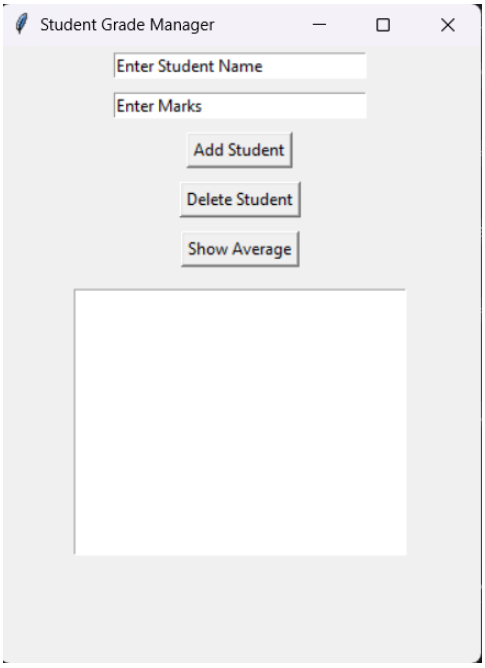
ii) GUI (Graphical User Interface)

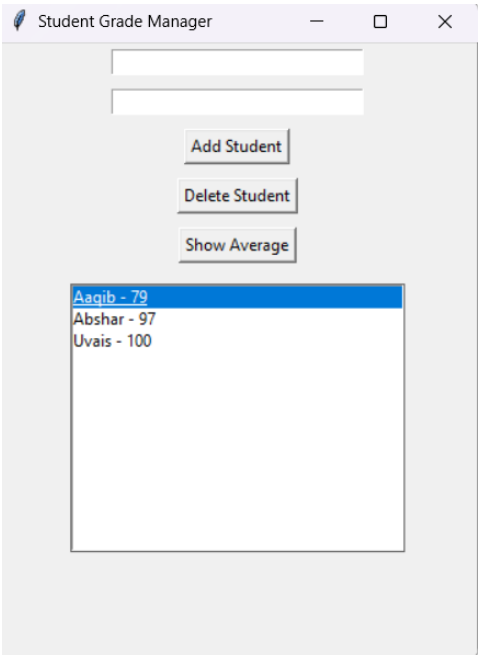
This Python program uses **Tkinter** to create a **GUI Student Grade Manager** that lets users add, view, delete students and calculate the average of their marks.

```
import tkinter as tk
from tkinter import messagebox

students = []

def add_student():
    name = name_entry.get()
    marks = marks_entry.get()

    if name and marks:
        try:
            marks = int(marks)
            students.append({"name": name, "marks": marks})
            name_entry.delete(0, tk.END)
            marks_entry.delete(0, tk.END)
            update_list()
        except ValueError:
            messagebox.showerror("Error", "Marks must be a number")
    else:
        messagebox.showwarning("Warning", "All fields are required")

def update_list():
    student_list.delete(0, tk.END)
    for s in students:
        student_list.insert(tk.END, f"{s['name']} - {s['marks']}")

def delete_student():
    selected = student_list.curselection()
    if selected:
        students.pop(selected[0])
        update_list()
    else:
        messagebox.showwarning("Warning", "Select a student to delete")

def show_average():
    if students:
        avg = sum(s["marks"] for s in students) / len(students)
        messagebox.showinfo("Average Marks", f"Average: {avg:.2f}")
    else:
        messagebox.showwarning("Warning", "No data available")

# App window
app = tk.Tk()
```

```python
app.title("Student Grade Manager")
app.geometry("350x450")

# Inputs
name_entry = tk.Entry(app, width=30)
name_entry.pack(pady=5)
name_entry.insert(0, "Enter Student Name")

marks_entry = tk.Entry(app, width=30)
marks_entry.pack(pady=5)
marks_entry.insert(0, "Enter Marks")

# Buttons
add_btn = tk.Button(app, text="Add Student", command=add_student)
add_btn.pack(pady=5)

delete_btn = tk.Button(app, text="Delete Student", command=delete_student)
delete_btn.pack(pady=5)

avg_btn = tk.Button(app, text="Show Average", command=show_average)
avg_btn.pack(pady=5)

# Listbox
student_list = tk.Listbox(app, width=40, height=12)
student_list.pack(pady=10)

# Run app
app.mainloop()
```

Sample Output:





iii) VUI (Voice User Interface)

This Python program is a **voice-controlled weather assistant** that listens to a city name using speech recognition, fetches weather data from the OpenWeather API, and speaks the result using text-to-speech.

```
import speech_recognition as sr
import pyttsx3
import requests

# Voice setup
recognizer = sr.Recognizer()
engine = pyttsx3.init()

API_KEY = "05de1c151ab811c2a0e695c375dbbfa0"  # <-- replace with your OpenWeather
API key

def speak(text):
    engine.say(text)
    engine.runAndWait()

def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("You said:", command)
        return command.lower()
    except sr.UnknownValueError:
        speak("Sorry, I didn't understand")
        return None

def get_weather(city):
    url =
f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metr
ic"

    try:
        response = requests.get(url)
        data = response.json()

        if data["cod"] != 200:
            speak("City not found")
            return

        temp = data["main"]["temp"]
```

```python
        desc = data["weather"][0]["description"]

        speak(f"The temperature in {city} is {temp} degree celsius with {desc}")

    except:
        speak("Error fetching weather data")

def main():
    speak("Voice Weather Assistant started")

    while True:
        speak("Say a city name or say exit to quit")
        command = recognize_speech()

        if not command:
            continue

        if "exit" in command:
            speak("Goodbye")
            break

        elif "weather in" in command:
            city = command.replace("weather in", "").strip()
            get_weather(city)

        else:
            speak("Please say weather in followed by city name")

if __name__ == "__main__":
    main()
```

Sample Output:

```
(Voice) Say a city name or say exit to quit
(User) weather in London
(Voice) The temperature in London is 15 degre celsius with
clear sky.
```

```
(Voice) Say a city name or say exit to quit
(User) exit
(Voice) Goodbye
```